Lecture 2 – Word2Vec

Xiaocheng Li
Imperial College Business School, Imperial College London

Introduction: Why Word Vectors?

The Problem of Word Meaning

How should we represent the meaning of a word?

- Traditional approach: Words as discrete symbols
- `dog`, `cat`, `animal` – just strings of letters
- In formal logic: DOG, CAT, ANIMAL

The Problem of Word Meaning

How should we represent the meaning of a word?

- Traditional approach: Words as discrete symbols
- `dog`, `cat`, `animal` – just strings of letters
- In formal logic: DOG, CAT, ANIMAL

Barbara Partee's Famous Joke

Q: What's the meaning of life?

A: LIFE

The Problem of Word Meaning

How should we represent the meaning of a word?

- Traditional approach: Words as discrete symbols
- `dog`, `cat`, `animal` – just strings of letters
- In formal logic: DOG, CAT, ANIMAL

Barbara Partee's Famous Joke

Q: What's the meaning of life?

A: LIFE

Surely we can do better than this!

What We Want from Word Meaning

A good model of word meaning should capture:

- **Similarity:** *cat* is similar to *dog*
- **Antonymy:** *cold* is opposite of *hot*
- **Connotations:** *happy* (positive) vs. *sad* (negative)
- **Semantic Relations:** *buy*/*sell*/*pay* are related perspectives on commerce
- **Inference:** Support reasoning for QA, dialogue, etc.

What We Want from Word Meaning

A good model of word meaning should capture:

- **Similarity:** *cat* is similar to *dog*
- **Antonymy:** *cold* is opposite of *hot*
- **Connotations:** *happy* (positive) vs. *sad* (negative)
- **Semantic Relations:** *buy*/*sell*/*pay* are related perspectives on commerce
- **Inference:** Support reasoning for QA, dialogue, etc.

The Distributional Hypothesis

"You shall know a word by the company it keeps" – J.R. Firth (1957)

Words that occur in similar contexts tend to have similar meanings.

The Distributional Hypothesis in Action

Unknown Word Example

Suppose you see the word *ongchoi* in these contexts:

- "Ongchoi is delicious sautÃ©ed with garlic"
- "Ongchoi is superb over rice"
- "...ongchoi leaves with salty sauces..."

The Distributional Hypothesis in Action

Unknown Word Example

Suppose you see the word *ongchoi* in these contexts:

- "Ongchoi is delicious sautÃ©ed with garlic"
- "Ongchoi is superb over rice"
- "...ongchoi leaves with salty sauces..."

And you've seen similar contexts for known words:

- "...spinach sautÃ©ed with garlic over rice..."
- "...chard stems and leaves are delicious..."
- "...collard greens and other salty leafy greens"

The Distributional Hypothesis in Action

Unknown Word Example

Suppose you see the word *ongchoi* in these contexts:

- "Ongchoi is delicious sautÃ©ed with garlic"
- "Ongchoi is superb over rice"
- "...ongchoi leaves with salty sauces..."

And you've seen similar contexts for known words:

- "...spinach sautÃ©ed with garlic over rice..."
- "...chard stems and leaves are delicious..."
- "...collard greens and other salty leafy greens"

Conclusion: *ongchoi* is probably a leafy green vegetable!

From Philosophy to Vectors

Osgood's Revolutionary Idea (1957)

Represent word meaning as a point in multidimensional space!

Example: Affective meaning dimensions

Word	Valence	Arousal	Dominance
courageous	8.05	5.5	7.38
music	7.67	5.57	6.5
heartbreak	2.45	5.65	3.58
cub	6.71	3.95	4.24

Each word = vector = point in space = $[2.45, 5.65, 3.58]$ for "heartbreak"

Traditional Approaches and Their Limitations

Traditional NLP: Words as Discrete Symbols

One-Hot Representation

Represent each word as a vector with:

- Dimension = vocabulary size (e.g., 50,000)
- Exactly one 1, rest are 0s

$\texttt{motel} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$
$\texttt{hotel} = [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$

Traditional NLP: Words as Discrete Symbols

One-Hot Representation

Represent each word as a vector with:

- Dimension = vocabulary size (e.g., 50,000)
- Exactly one 1, rest are 0s

$$\texttt{motel} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$$
$$\texttt{hotel} = [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$$

Major Problem

No notion of similarity!

- All pairs of words are equally distant
- $\texttt{hotel} \cdot \texttt{motel} = 0$ (orthogonal)
- Can't capture that "hotel" and "motel" are similar

WordNet and Its Limitations

WordNet: A Thesaurus Approach

- Lists of synonym sets and hypernyms
- Hand-crafted by linguists
- Example: "good" has 20+ different senses

WordNet and Its Limitations

WordNet: A Thesaurus Approach

- Lists of synonym sets and hypernyms
- Hand-crafted by linguists
- Example: "good" has 20+ different senses

Problems with WordNet

- **Missing nuance:** "proficient" $\neq$ "good" in all contexts
- **Missing new meanings:** wicked, badass, ninja, wizard (as "cool")
- **Subjective:** Human labor required
- **Incomplete:** Can't compute accurate similarities
- **Static:** Hard to keep up-to-date

Vector Semantics Foundations

Building Meaning from Co-occurrence

Term-Document Matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Building Meaning from Co-occurrence

Term-Document Matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- Each **row** = word vector (based on document occurrences)
- Each **column** = document vector (based on word occurrences)
- Similar words $\rightarrow$ similar vectors
- *fool* and *wit* co-occur in comedies!

Term-Term (Word-Context) Matrices

Context Window Approach

Count co-occurrences within a window (e.g., ± 4 words):

	aardvark	...	computer	data	pie	sugar
cherry	0	...	2	8	442	25
strawberry	0	...	0	0	60	19
digital	0	...	1670	1683	5	4
information	0	...	3325	3982	5	13

Term-Term (Word-Context) Matrices

Context Window Approach

Count co-occurrences within a window (e.g., ± 4 words):

	aardvark	...	computer	data	pie	sugar
cherry	0	...	2	8	442	25
strawberry	0	...	0	0	60	19
digital	0	...	1670	1683	5	4
information	0	...	3325	3982	5	13

Observations:

- *cherry* and *strawberry* both occur with *pie* and *sugar*
- *digital* and *information* both occur with *computer* and *data*
- Context reveals semantic similarity!

Measuring Similarity: Cosine

Dot Product

$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{n} v_i w_i$

Problem: Favors longer vectors (frequent words)

Measuring Similarity: Cosine

Dot Product

$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{n} v_i w_i$

Problem: Favors longer vectors (frequent words)

Cosine Similarity

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{n} v_i w_i}{\sqrt{\sum_{i=1}^{n} v_i^2}\sqrt{\sum_{i=1}^{n} w_i^2}}$$

- Normalized dot product
- Range: $[-1, 1]$ (or $[0, 1]$ for count vectors)
- $\cos = 1$: identical direction
- $\cos = 0$: orthogonal

TF-IDF and Weighting

Problems with Raw Frequency Counts

Two Conflicting Intuitions

1. **Frequent co-occurrence matters:** *pie* near *cherry* is important
2. **Too frequent is uninformative:** *the*, *it*, *good* occur everywhere

Problems with Raw Frequency Counts

Two Conflicting Intuitions

1. **Frequent co-occurrence matters:** *pie* near *cherry* is important
2. **Too frequent is uninformative:** *the*, *it*, *good* occur everywhere

Example: In Shakespeare corpus, *good* appears in all 37 plays

- High frequency everywhere
- Not discriminative between comedies vs. tragedies
- We need to downweight ubiquitous words

Problems with Raw Frequency Counts

Two Conflicting Intuitions

1. **Frequent co-occurrence matters:** *pie* near *cherry* is important
2. **Too frequent is uninformative:** *the*, *it*, *good* occur everywhere

Example: In Shakespeare corpus, *good* appears in all 37 plays

- High frequency everywhere
- Not discriminative between comedies vs. tragedies
- We need to downweight ubiquitous words

Solution Preview

TF-IDF: Balance term frequency with inverse document frequency

TF-IDF: Term Frequency $\times$ Inverse Document Frequency

Term Frequency (TF)

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10}(\mathrm{count}(t,d)) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

TF-IDF: Term Frequency $\times$ Inverse Document Frequency

Term Frequency (TF)

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10}(\mathrm{count}(t,d)) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Inverse Document Frequency (IDF)

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

where N = total documents, df_t = documents containing term t

TF-IDF: Term Frequency $\times$ Inverse Document Frequency

Term Frequency (TF)

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10}(\mathrm{count}(t,d)) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Inverse Document Frequency (IDF)

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

where N = total documents, df_t = documents containing term t

TF-IDF Weight

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

TF-IDF Example: Shakespeare Corpus

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.97
forest	12	0.49
battle	21	0.25
wit	34	0.04
fool	36	0.01
good	37	0.00

Key insight: Rare words like "Romeo" get high weights, common words like "good" get zero weight!

Alternative: Positive Pointwise Mutual Information

Pointwise Mutual Information (PMI)

$$\mathrm{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

Measures how much more w and c co-occur than expected by chance

Alternative: Positive Pointwise Mutual Information

Pointwise Mutual Information (PMI)

$$\mathrm{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

Measures how much more w and c co-occur than expected by chance

Positive PMI (PPMI)

$$\mathrm{PPMI}(w, c) = \max\left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0\right)$$

Replace negative PMI values with 0 (unreliable for small corpora)

Alternative: Positive Pointwise Mutual Information

Pointwise Mutual Information (PMI)

$$\mathrm{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

Measures how much more w and c co-occur than expected by chance

Positive PMI (PPMI)

$$\mathrm{PPMI}(w, c) = \max\left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0\right)$$

Replace negative PMI values with 0 (unreliable for small corpora)

Used for term-term matrices; TF-IDF used for term-document matrices

Logistic Regression Primer

Logistic Regression: Key to Word2Vec

Binary Classification Problem

Given features $\mathbf{x}$, predict probability of class $y = 1$:

$$P(y = 1|\mathbf{x}) = ?$$

Logistic Regression: Key to Word2Vec

Binary Classification Problem

Given features $\mathbf{x}$, predict probability of class $y = 1$:

$$P(y = 1|\mathbf{x}) = ?$$

Linear Model Problem

$$P(y = 1|\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

Problem: Right side can be any real number, but probability must be in $[0, 1]$!

Logistic Regression: Key to Word2Vec

Binary Classification Problem

Given features $\mathbf{x}$, predict probability of class $y = 1$:

$$P(y = 1|\mathbf{x}) = ?$$

Linear Model Problem

$$P(y = 1|\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

Problem: Right side can be any real number, but probability must be in $[0, 1]$!

Solution: Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

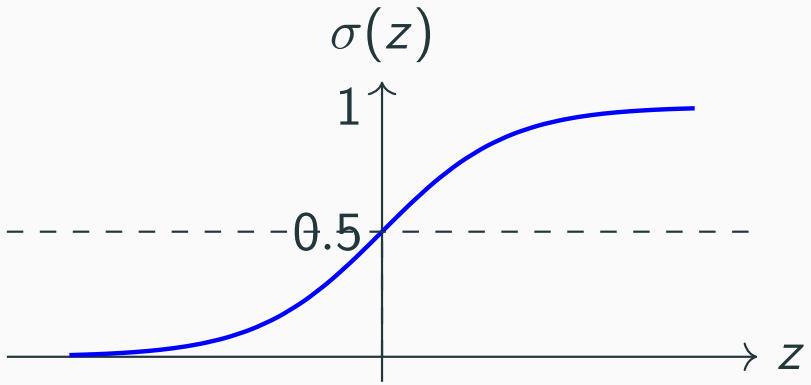
Maps $(-\infty, \infty) \to (0, 1)$

The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- $\sigma(0) = 0.5$
- $\sigma(z) \to 1$ as $z \to \infty$
- $\sigma(z) \to 0$ as $z \to -\infty$
- Derivative:
  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$



Logistic Regression Model

Probability Model

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

$$P(y = 0|\mathbf{x}) = 1 - P(y = 1|\mathbf{x}) = \sigma(-(\mathbf{w}^T\mathbf{x} + b))$$

Logistic Regression Model

Probability Model

$$P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}+b) = \frac{1}{1+e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

$$P(y=0|\mathbf{x}) = 1 - P(y=1|\mathbf{x}) = \sigma(-(\mathbf{w}^T\mathbf{x}+b))$$

Training: Maximum Likelihood

Given training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, maximize:

$$\mathcal{L} = \prod_{i=1}^{n} P(y_i|\mathbf{x}_i)$$

Equivalent to minimizing negative log-likelihood:

$$J = -\sum_{i=1}^{n} \log P(y_i|\mathbf{x}_i)$$

Training via Gradient Descent

Cross-Entropy Loss

$$J = -\sum_{i=1}^{n} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$

Training via Gradient Descent

Cross-Entropy Loss

$$J = -\sum_{i=1}^{n} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$

Gradients

$$\frac{\partial J}{\partial \mathbf{w}} = \sum_{i=1}^{n} (\hat{y}_i - y_i) \mathbf{x}_i$$

$$\frac{\partial J}{\partial b} = \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

Training via Gradient Descent

Cross-Entropy Loss

$$J = -\sum_{i=1}^{n} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i + b)$

Gradients

$$\frac{\partial J}{\partial \mathbf{w}} = \sum_{i=1}^{n} (\hat{y}_i - y_i) \mathbf{x}_i$$

$$\frac{\partial J}{\partial b} = \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

Update Rules

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

Word2Vec: Skip-gram Model

Word2Vec: The Big Idea

Core Intuition

Instead of counting co-occurrences, **predict** them!

Train a classifier: "Is word w likely to appear near word c?"

- We don't care about the classification task itself
- We want the learned weights as our word embeddings!

Word2Vec: The Big Idea

Core Intuition

Instead of counting co-occurrences, **predict** them!

Train a classifier: "Is word w likely to appear near word c?"

- We don't care about the classification task itself
- We want the learned weights as our word embeddings!

Self-Supervision

Use running text as training data:

- Positive examples: actual word-context pairs
- Negative examples: random word-context pairs
- No manual labeling required!

Word2Vec: The Big Idea

Core Intuition

Instead of counting co-occurrences, **predict** them!

Train a classifier: "Is word w likely to appear near word c?"

- We don't care about the classification task itself
- We want the learned weights as our word embeddings!

Self-Supervision

Use running text as training data:

- Positive examples: actual word-context pairs
- Negative examples: random word-context pairs
- No manual labeling required!

Revolutionary insight: Transform an unsupervised problem into supervised learning!

Skip-gram Model Setup

... lemon, a $\boxed{\text{tablespoon of apricot jam, a}}$ pinch ...

Two Types of Embeddings

For each word w in vocabulary V:

- **Target embedding** $\mathbf{v}_w$ (when w is the center word)
- **Context embedding** $\mathbf{u}_w$ (when w is in the context)

Skip-gram Model Setup

... lemon, a $\boxed{\text{tablespoon of apricot jam, a}}$ pinch ...

Two Types of Embeddings

For each word w in vocabulary V:

- **Target embedding $\mathbf{v}_w$** (when w is the center word)
- **Context embedding $\mathbf{u}_w$** (when w is in the context)

Training Data

Positive examples: (apricot, tablespoon), (apricot, of), (apricot, jam), (apricot, a)

Negative examples: (apricot, aardvark), (apricot, democracy), ...

Skip-gram Probability Model

Probability of Context Word Given Center Word

$$P(o|c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)}$$

where:

- c = center word, o = outside (context) word
- $\mathbf{v}_c$ = target embedding of center word
- $\mathbf{u}_o$ = context embedding of outside word

Skip-gram Probability Model

Probability of Context Word Given Center Word

$$P(o|c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)}$$

where:

- c = center word, o = outside (context) word
- $\mathbf{v}_c$ = target embedding of center word
- $\mathbf{u}_o$ = context embedding of outside word

Softmax Function

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{n} \exp(x_j)}$$

- Maps real values to probability distribution
- Larger $\mathbf{u}_o^T \mathbf{v}_c \rightarrow$ higher probability
- Normalizes over entire vocabulary

Skip-gram Objective Function

Data Likelihood

For position t with window size m:

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

Skip-gram Objective Function

Data Likelihood

For position t with window size m:

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

Objective Function (Negative Log-Likelihood)

$$J(\theta) = -\frac{1}{T} \log \mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Skip-gram Objective Function

Data Likelihood

For position t with window size m:

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

Objective Function (Negative Log-Likelihood)

$$J(\theta) = -\frac{1}{T} \log \mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Goal: Minimize $J(\theta)$ = Maximize predictive accuracy!

θ represents all parameters: $\{\mathbf{v}_w, \mathbf{u}_w\}_{w \in V}$

Skip-gram with Negative Sampling (SGNS)

Computational Problem

Softmax denominator requires summing over entire vocabulary:

$$\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)$$

For $|V| = 50,000$, this is expensive!

Skip-gram with Negative Sampling (SGNS)

Computational Problem

Softmax denominator requires summing over entire vocabulary:

$$\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)$$

For $|V| = 50,000$, this is expensive!

Negative Sampling Solution

For each positive example (w, c_{pos}), create k negative examples (w, c_{neg})

New objective for one word-context pair:

$$\log \sigma(\mathbf{u}_{c_{pos}}^T \mathbf{v}_w) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{c_{neg_i}}^T \mathbf{v}_w)$$

Skip-gram with Negative Sampling (SGNS)

Computational Problem

Softmax denominator requires summing over entire vocabulary:

$$\sum_{w \in V} \exp(\mathbf{u}_w^T \mathbf{v}_c)$$

For $|V| = 50,000$, this is expensive!

Negative Sampling Solution

For each positive example (w, c_{pos}), create k negative examples (w, c_{neg})

New objective for one word-context pair:

$$\log \sigma(\mathbf{u}_{c_{pos}}^T \mathbf{v}_w) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{c_{neg_i}}^T \mathbf{v}_w)$$

Intuition: Maximize similarity to true contexts, minimize similarity to random contexts

Negative Sampling: Implementation Details

How to Choose Negative Examples?

Sample noise words according to unigram frequency raised to power $\alpha = 0.75$:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w' \in V} \text{count}(w')^\alpha}$$

Negative Sampling: Implementation Details

How to Choose Negative Examples?

Sample noise words according to unigram frequency raised to power $\alpha = 0.75$:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w' \in V} \text{count}(w')^\alpha}$$

Why $\alpha = 0.75$?

- $\alpha = 1$: Proportional to frequency (favors "the", "a", "is")
- $\alpha = 0$: Uniform distribution
- $\alpha = 0.75$: Sweet spot that gives rare words higher probability

Example: If $P(\text{"the"}) = 0.1$ and $P(\text{"aardvark"}) = 0.0001$

- $P_{0.75}(\text{"the"}) \approx 0.032$
- $P_{0.75}(\text{"aardvark"}) \approx 0.0018$

Word2Vec Training Algorithm

Skip-gram with Negative Sampling Training

Initialization: Random embeddings $\mathbf{v}_w, \mathbf{u}_w$ for all $w \in V$

For each training epoch:

- For each word w_t in corpus
- For each context word w_{t+j} in window
- Sample k negative words $\{w_{neg_i}\}$
- Compute gradients for $\mathbf{v}_{w_t}, \mathbf{u}_{w_{t+j}}, \{\mathbf{u}_{w_{neg_i}}\}$
- Update embeddings using gradient descent

Return: Final embeddings $\{\mathbf{v}_w\}$ or $\{\mathbf{v}_w + \mathbf{u}_w\}$

Word2Vec: Gradient Computation

SGNS Loss for One Example

$$L = -\log \sigma(\mathbf{u}_{c_{pos}}^T \mathbf{v}_w) - \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{c_{neg_i}}^T \mathbf{v}_w)$$

Word2Vec: Gradient Computation

SGNS Loss for One Example

$$L = -\log \sigma(\mathbf{u}_{c_{pos}}^T \mathbf{v}_w) - \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{c_{neg_i}}^T \mathbf{v}_w)$$

Gradients

$$\frac{\partial L}{\partial \mathbf{v}_w} = [\sigma(\mathbf{u}_{c_{pos}}^T \mathbf{v}_w) - 1]\mathbf{u}_{c_{pos}} + \sum_{i=1}^{k} \sigma(\mathbf{u}_{c_{neg_i}}^T \mathbf{v}_w)\mathbf{u}_{c_{neg_i}}$$

$$\frac{\partial L}{\partial \mathbf{u}_{c_{pos}}} = [\sigma(\mathbf{u}_{c_{pos}}^T \mathbf{v}_w) - 1]\mathbf{v}_w$$

$$\frac{\partial L}{\partial \mathbf{u}_{c_{neg_i}}} = \sigma(\mathbf{u}_{c_{neg_i}}^T \mathbf{v}_w)\mathbf{v}_w$$

Properties and Applications

Amazing Properties of Word Embeddings

Vector Analogies

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

$$\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}} \approx \overrightarrow{\text{Rome}}$$

Amazing Properties of Word Embeddings

Vector Analogies

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

$$\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}} \approx \overrightarrow{\text{Rome}}$$

Similarity Clusters

Nearest neighbors of "frog": frogs, toad, litoria, leptodactylidae, rana, lizard...

Nearest neighbors of "January": February, March, April, month, year...

Amazing Properties of Word Embeddings

Vector Analogies

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

$$\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}} \approx \overrightarrow{\text{Rome}}$$

Similarity Clusters

Nearest neighbors of "frog": frogs, toad, litoria, leptodactylidae, rana, lizard...

Nearest neighbors of "January": February, March, April, month, year...

Context Window Effects

- **Small windows ($\pm$2):** Syntactic similarity (POS, morphology)
- **Large windows ($\pm$5):** Topical similarity (semantic fields)

Bias and Embeddings

Embeddings Reflect Training Data Biases

Examples from real embeddings:

- "computer programmer" - "man" + "woman" $\approx$ "homemaker"
- "father" : "doctor" :: "mother" : "nurse"
- African-American names associated with unpleasant words
- Male names associated with math, female names with arts

Bias and Embeddings

Embeddings Reflect Training Data Biases

Examples from real embeddings:

- "computer programmer" - "man" + "woman" $\approx$ "homemaker"
- "father" : "doctor" :: "mother" : "nurse"
- African-American names associated with unpleasant words
- Male names associated with math, female names with arts

Two Types of Harm

- **Allocational Harm:** Unfair resource allocation (jobs, credit)
- **Representational Harm:** Demeaning or ignoring social groups

Bias and Embeddings

Embeddings Reflect Training Data Biases

Examples from real embeddings:

- "computer programmer" - "man" + "woman" $\approx$ "homemaker"
- "father" : "doctor" :: "mother" : "nurse"
- African-American names associated with unpleasant words
- Male names associated with math, female names with arts

Two Types of Harm

- **Allocational Harm:** Unfair resource allocation (jobs, credit)
- **Representational Harm:** Demeaning or ignoring social groups

Current Research: Debiasing methods, but eliminating bias completely remains challenging

Applications and Evaluation

Applications

- **Information Retrieval:** Document similarity, query expansion
- **Machine Translation:** Cross-lingual embeddings
- **Sentiment Analysis:** Features for classification
- **Question Answering:** Semantic matching
- **Recommendation Systems:** Item similarity

Applications and Evaluation

Applications

- **Information Retrieval:** Document similarity, query expansion
- **Machine Translation:** Cross-lingual embeddings
- **Sentiment Analysis:** Features for classification
- **Question Answering:** Semantic matching
- **Recommendation Systems:** Item similarity

Evaluation Methods

- **Intrinsic:** Word similarity (WordSim-353, SimLex-999), analogies
- **Extrinsic:** Performance on downstream NLP tasks
- **Visualization:** t-SNE, hierarchical clustering

Applications and Evaluation

Applications

- **Information Retrieval:** Document similarity, query expansion
- **Machine Translation:** Cross-lingual embeddings
- **Sentiment Analysis:** Features for classification
- **Question Answering:** Semantic matching
- **Recommendation Systems:** Item similarity

Evaluation Methods

- **Intrinsic:** Word similarity (WordSim-353, SimLex-999), analogies
- **Extrinsic:** Performance on downstream NLP tasks
- **Visualization:** t-SNE, hierarchical clustering

Best Practice: Always evaluate on downstream tasks, not just intrinsic metrics!

Conclusion

Summary: From Symbols to Vectors

Key Insights

1. **Distributional Hypothesis:** Word meaning from context
2. **Vector Representation:** Dense, real-valued vectors capture similarity
3. **Self-Supervision:** Transform unsupervised $\rightarrow$ supervised learning
4. **Efficiency:** Skip-gram + negative sampling scales to large corpora

Summary: From Symbols to Vectors

Key Insights

1. **Distributional Hypothesis:** Word meaning from context
2. **Vector Representation:** Dense, real-valued vectors capture similarity
3. **Self-Supervision:** Transform unsupervised $\rightarrow$ supervised learning
4. **Efficiency:** Skip-gram + negative sampling scales to large corpora

Word2Vec Impact

- Revolutionized NLP (2013-2018)
- Foundation for modern deep learning in NLP
- Led to contextual embeddings (ELMo, BERT, GPT)
- Influenced representation learning across domains

Summary: From Symbols to Vectors

Key Insights

1. **Distributional Hypothesis:** Word meaning from context
2. **Vector Representation:** Dense, real-valued vectors capture similarity
3. **Self-Supervision:** Transform unsupervised $\rightarrow$ supervised learning
4. **Efficiency:** Skip-gram + negative sampling scales to large corpora

Word2Vec Impact

- Revolutionized NLP (2013-2018)
- Foundation for modern deep learning in NLP
- Led to contextual embeddings (ELMo, BERT, GPT)
- Influenced representation learning across domains

The remarkable result: Word meaning can be represented surprisingly well by vectors of real numbers!

Beyond Word2Vec: The Path Forward

Limitations of Static Embeddings

- One embedding per word (polysemy problem)
- No context-dependent meaning
- Fixed vocabulary (OOV problem)

Beyond Word2Vec: The Path Forward

Limitations of Static Embeddings

- One embedding per word (polysemy problem)
- No context-dependent meaning
- Fixed vocabulary (OOV problem)

Modern Extensions

- **FastText:** Subword embeddings for morphology
- **GloVe:** Global matrix factorization approach
- **Contextual Embeddings:** ELMo, BERT, GPT
- **Transformer Models:** Attention-based architectures

Beyond Word2Vec: The Path Forward

Limitations of Static Embeddings

- One embedding per word (polysemy problem)
- No context-dependent meaning
- Fixed vocabulary (OOV problem)

Modern Extensions

- **FastText:** Subword embeddings for morphology
- **GloVe:** Global matrix factorization approach
- **Contextual Embeddings:** ELMo, BERT, GPT
- **Transformer Models:** Attention-based architectures

Current State (2025)

While transformers dominate, Word2Vec principles still underlie modern NLP:

- Self-supervised learning from text
- Vector representations of meaning

Questions?

Key Takeaways

- Word vectors capture semantic similarity
- Self-supervision from raw text is powerful
- Skip-gram + negative sampling = efficient training
- Embeddings enable modern NLP applications
- Bias considerations are crucial

Thank you for your attention!